BIL 322 · DÖNEM PROJESİ · FAZ 1 LABORATUVARI

Otonom Drone Sürüsü Koreografları

Thread’ler, kuyruklar, kilitler, barrier’lar ve bataryası biten bir kaptan.



NOTLAR

Bu slaytlar, dönem projesinin Faz 1 laboratuvar sunumudur. Her slaytın altındaki notlar senin içindir: laboratuvardan sonra Faz 1 föyü ve proje açıklamasıyla birlikte oku. Dönem projesi, her biri yaklaşık bir hafta süren üç fazdan oluşur. Faz 1, thread ve senkronizasyon kısmıdır. Renkli noktalar drone'lardır; projenin sonunda gökyüzünde bir resim oluştururlar.

PROJE

Müzikle dans eden bir drone sürüsü

Yüzlerce LED drone bir koreografi sergiler, sonra gökyüzünde bir resim oluşturur. Gösteriyi dört rol yapar.

Merkez

Sanat Yönetmeni

Notayı ve hedef resmi okur, her bölümün temasını belirler, beat saatini çalıştırır, yavaş planlayıcıyı barındırır.

Leader'lar

Dans Kaptanları

Her bölge için bir süreç. Her biri plan ister, planı doğrular ve topluluğunu yönetir.

Follower'lar

Dansçılar

Faz 1'de bir leader'ın içindeki thread'ler. Her biri bir hücreye uçar, bir renk yakar, rapor verir.

Viewer

Yayın Kabini

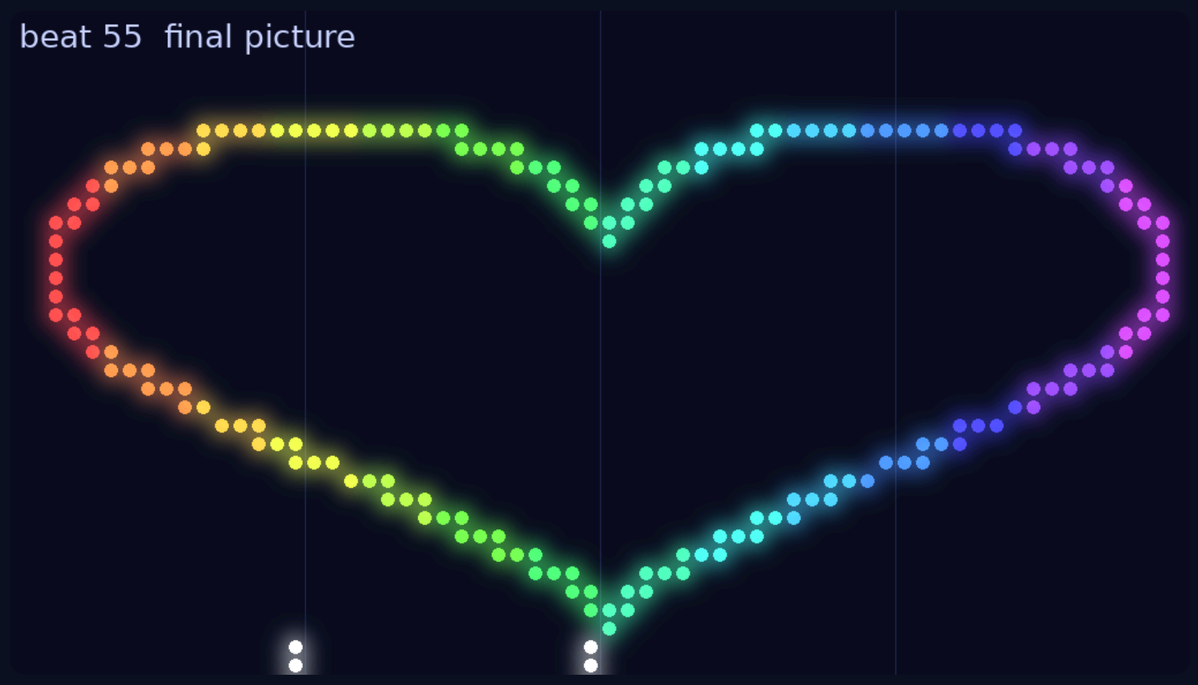
Faz 2'den itibaren ayrı bir GUI. Kare atlayabilir, sürüyü asla yavaşlatmaz.

NOTLAR

Hikâye: mühendislik fakültesi bir stadyum geleneğini havaya taşıyor. Merkez (center), notayı (müziğin bölümleri ve ruh hâlleri) ve hedef resmi bilir; her bölüm için yalnızca genel temayı belirler. Ayrıntılı hareketler, yavaş bir düşünür olan koreograftan gelir. Gerçek bir gösteride bu bir LLM olurdu; burada uyuyan bir taklit (stub) olduğu için yavaş, meşgul ya da yanlış bir modeli beklemenin nasıl bir şey olduğunu hissedersin. Leader'lar dans kaptanlarıdır, her bölge için bir süreç. Follower'lar ucuz dansçılardır; Faz 1'de kaptanlarının içindeki thread'lerdir. Viewer, Faz 2'de ayrı bir GUI olur. Bugün yalnızca sahnenin metin olarak çizilmiş hâlidir.

HEDEF

Çalışınca nasıl görünür



4 kaptan × 46 dansçı = **184 drone**. Her nokta bir drone'dur, kalbin her pikseline bir drone düşer.

Alttaki 4 beyaz nokta, evde kalan yedek drone'lardır.

```
./center --score data/scores/show.score --
followers 46 --ascii 8
```

Faz 2 bunu gerçek bir pencerede çizer.

NOTLAR

Bu, Faz 1'in nihai hedefidir ve center'ın metin çıktısından çizilmiştir. Animasyon tüm gösteriyi oynatır: tüm drone'lar altta kendi başlangıç sıralarında (her bölge bir kaptanın topluluğudur) başlar, sonra bir dalga formasyonu gelir, sonra drone'ların hedef resmin (gökkuşağı renkli bir kalp) piksellerine uçtuğu final gelir. Sayılar: 4 leader çarpı 46 follower, 180 piksel için 184 drone eder; dört yedek drone evde kalır. Komut, görevlerin çalıştıktan sonra bunu yeniden üretir. O zamana kadar gösteri uçtan uca çalışmaz. Faz 2 aynı şeyi gerçek bir pencerede (SDL, GTK veya Qt) çizer.

ÜÇ FAZ, TEK SİSTEM

Aynı protokol, üç taşıma yolu

	Faz 1 (bu hafta)	Faz 2	Faz 3
Leader	child süreç (fork, exec)	child süreç	client süreç
Dansçı	kaptanın içinde thread	kaptanın içinde thread	ayrı süreç
Bağlantılar	pipe’lar	pipe’lar + paylaşımlı bellek	TCP socket’ler
Viewer	terminalde metin	ayrı süreç, paylaşımlı bellek	socket client
Kaptan ölünce	center fark eder ve yeniden başlatır	+ hücreleri serbest bırakılır	yetim kalanlar yeniden yerleştirilir

Gönderme ve alma kodunu **file descriptor**’lar üzerine yaz ve ayrıştırmayı (parsing) taşıma yolundan ayrı tut: Faz 3’te programı değil, yalnızca taşıma yolunu değiştirirsin.

NOTLAR

Protokol tüm fazlarda aynı kalır; değişen, parçaların birbiriyle nasıl konuştuğudur. Bu hafta leader'lar fork ve exec ile oluşturulan child süreçlerdir (Lab 4 kalıbın) ve pipe'larla bağlanırlar (Lab 5). Dansçılar thread'dir. Faz 2'de sahne paylaşımlı belleğe, viewer ise kendi sürecine taşınır. Faz 3'te her şey socket kullanır ve dansçılar ayrı süreçler olur; böylece bir kaptan öldüğünde topluluğu gerçekten yetim kalır. Tüm projenin arkasındaki soru: bir kaptan öldüğünde bunu kim fark eder ve topluluğunu kim devralır? Kodun zaten file descriptor'lar üzerinden satır gönderdiği için daha sonra taşıma yolunu değiştirmek küçük bir iştir.

FAZ 1 MİMARİSİ

İki tür süreç, çok sayıda thread

center

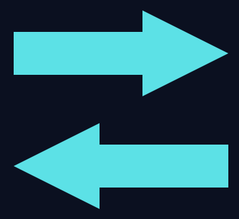
tek süreç, parent

main thread · saat döngüsü: THEME, BEAT, READY ve ACK bekler, sahneyi denetler

reader thread'ler · her leader için bir tane, pipe'ını ayrıştırır

planner havuzu · sınırlı bir istek kuyruğunun arkasında thread'ler, Lab 6 havuzu + arena

pipe'lar



mesaj başına bir metin satırı

leader × 4

bölge başına bir child süreç: fork, dup2, exec

main thread · center'ın satırlarını okur

thinker · PLAN_REQ, doğrulama, READY

follower thread'ler · F0 … Fk, beat başına bir barrier, bölgenin doluluk ızgarasında hücre talep eder

NOTLAR

Center'ın main thread'i saati çalıştırır: her bölümün temasını ve her beat'i gönderir; her leader bir bölüm için READY, her beat için ACK diyene kadar bekler. Her leader için bir reader thread, o leader'ın pipe'ını ayrıştırır. Koreograf, sınırlı bir istek kuyruğunun arkasındaki bir thread havuzudur ve her istek belleğini Lab 6 havuzundan alır. Her leader, Lab 4 kabuğundaki gibi fork, dup2 ve exec ile başlatılan bir child süreçtir. İçinde: pipe'ı okuyan bir main thread, plan isteyen bir thinker thread'i ve her follower için bir thread. Doluluk ızgarası leader başınadır, çünkü follower'lar onun thread'idir ve hücreleri doğrudan talep edebilir. Center leader'lara güvenmez: çarpışma ve hız için POS raporlarını denetler. Kendi kısmını yazmadan önce center.c'yi bir kez baştan sona oku; kilitleri ve sıraları yorumlarla açıklanmıştır.

PROTOKOL (SABİT)

Satır başına bir mesaj, düz metin

center → leader

`ASSIGN id x0 y0 x1 y1 n`	senin bölgen
`THEME sec start end theme mood`	genel yön
`PLAN req n`	ardından n keyframe satırı
`BUSY req retry_ms`	kuyruk dolu, sonra dene
`BEAT n`	n. beat'i başlat
`SHUTDOWN`	hoşça kal

leader → center

`HELLO id n`	buradayım
`PLAN_REQ req sec n seed`	bana bir plan ver
`READY id sec`	plan yüklendi
`POS id beat f x y c`	f nerede
`ACK id beat`	beat bitti

- Keyframe satırı: **`beat follower x y color`**. Bir plan, her beat ve her follower için bir tane içerir.
- Bilinmeyen mesaj: logla ve yok say. Bozuk mesaj: reddet, asla çökme. Satırlar en fazla 256 byte.
- Bu satırları stdin'ine elle yazarak bir leader'ı test edebilirsin.

NOTLAR

Protokol sabit bir sözleşmedir. Her mesaj düz metin bir satırdır; bu yüzden hem insanlar hem ajanlar okuyabilir: bir provanın akışını izleyebilir ya da test etmek için bir leader'ın stdin'ine elle mesaj yazabilirsin. Center'dan: ASSIGN bir leader'a bölgesini verir, THEME bir bölümün genel yönünü verir, PLAN ayrıntılı koreografiyi n keyframe satırı olarak taşır, BUSY istek kuyruğunun dolu olduğunu söyler, BEAT saattir, SHUTDOWN gösteriyi bitirir. Leader'dan: HELLO, plan sipariş etmek için PLAN_REQ, plan yüklenince READY, her beat'ten sonra her follower'ın nerede olduğunu bildirmek için POS ve ACK. Plan yoğundur (dense): her beat ve her follower için bir keyframe. Bilinmeyen mesajlar loglanır ve yok sayılır; bozuk olanlar reddedilir ve alıcıyı asla çökertmez. Ayrıştırıcı hazır verilir; sen onu kullanırsın. Proje açıklamasının 3. bölümüne bak.

NASIL ÇALIŞIR

Bir beat, adım adım

1. Center **BEAT b** gönderir (ancak b−1 için ACK geldikten sonra).
2. Her follower, kayıtlı plandan b için hedefini bulur (plan yoksa: olduğu yerde kalır).
3. Bulunduğun hücreyi **bırak (release)**, sonra **barrier**'da bekle.
4. Hedef hücreyi ızgarada **talep et (claim)**, sonra tekrar **barrier**.
5. Barrier'ın seçtiği thread **POS** satırlarını ve **ACK b**'yi gönderir.
6. Center çarpışmaları ve hızı denetler, sonra BEAT b+1 gönderir.

Neden iki barrier?

Bir beat içinde A follower'ı, B follower'ının terk ettiği hücreye girebilir.

Tüm bırakmalar herhangi bir talepten önce olmalı, yoksa A'nın talebi yanlışlıkla başarısız olur.

Başarısız bir talep bir güvenlik ihlalidir; döngüde tekrar denenecek bir şey değildir.

NOTLAR

Bu, Görev 3 ve 5'in kalbidir. Center her leader'a BEAT b gönderir ve b için ACK gelmeden asla b+1 göndermez: herkesi adım adım senkron tutan budur. Leader'da main thread beat'i yayımlar; her follower uyanır, hedefini kayıtlı plandan bulur, plan yoksa olduğu yerde kalır. Sonra barrier'larla ayrılmış iki aşama. Aşama 1: herkes bulunduğu hücreyi bırakır; barrier. Aşama 2: herkes hedef hücresini talep eder; barrier. Neden iki? Follower'lar aynı anda hareket eder. A, B'nin terk ettiği hücreyi isterse ve B henüz bırakmadıysa A'nın talebi başarısız olurdu. Arada bir barrier olunca tüm bırakmalar herhangi bir talepten önce gerçekleşir. İkinci barrier'dan sonra pthread_barrier_wait tam olarak bir thread'e PTHREAD_BARRIER_SERIAL_THREAD döndürür; o thread tüm follower'lar için POS satırlarını ve ardından ACK'yi gönderir. Başarısız bir talep bir güvenlik ihlalidir, döngüde tekrar denenecek bir şey değildir. Düşün: bir follower erken return edip bir barrier'ı atlarsa ne olur? Diğer herkes sonsuza kadar bekler.

YAPMAN GEREKENLER

İskelette altı görev

#	Görev	Neresi	Puan
1	Sınırlı kuyruk	`queue.c`	20%
2	Koreograf ve kaptan	`center.c, leader.c (thinker)`	25%
3	Dansçılar ve ızgara	`grid.c, leader.c (follower)`	25%
4	Deadlock'u bul	`grid_swap, swapstress`	10%
5	Kaptan düştü	`center.c (on_leader_exit)`	10%
6	Testler ve rapor	`make test, make tsan, REPORT.md`	10%

Hazır verilenler: protokol ayrıştırıcısı, planlayıcı, doğrulayıcı, yükleyiciler, saat döngüsü, birim testleri ve test betiği. **Senin işin:** TODO blokları.

NOTLAR

Bütün ödev tek sayfada. İskelet repository'si protokol ayrıştırıcısını, planlayıcıyı (LLM'in yavaş taklidi), plan doğrulayıcısını, score ve resim yükleyicilerini, center'ın saat döngüsünü ve testleri zaten içerir. Sen TODO bloklarını doldurursun. Görev 1: sınırlı kuyruk. Görev 2: center'daki istek hattı ve leader'daki thinker. Görev 3: iki kilit modunda doluluk ızgarası ve follower thread'leri. Görev 4: grid_swap içinde zaten bulunan bir deadlock; onu yeniden üretir, açıklar ve düzeltirsin. Görev 5: ölen bir kaptanı fark edip yeniden başlatmak. Görev 6: make test ve make tsan geçmeli, ayrıca kısa bir rapor. Her görevin ayrıntıları föyde.

GÖREV 1 · %20

Sınırlı kuyruk

```
queue_t *queue_create(size_t cap, size_t item_size);
int queue_push(queue_t *q, const void *item); // doluysa bekler
int queue_try_push(queue_t *q, const void *item); // doluysa -1
int queue_pop(queue_t *q, void *out); // boşsa bekler
void queue_close(queue_t *q); // herkesi uyandırır
size_t queue_size(queue_t *q); void queue_destroy(queue_t *q);
```

İki yerde kullanılır: center'ın **istek kuyruğu** (try_push, doluysa BUSY) ve leader'ın tema kuyruğu.

Kurallar

- Her pthread_cond_wait'in etrafında if değil **while**
- close'dan sonra: pop kalanları boşaltır, sonra −1 döner; push −1 döner
- close **herkesi** uyandırır: broadcast
- Meşgul bekleme (busy-waiting) yok, sleep yok

NOTLAR

Görev 1, derste gördüğün klasik sınırlı tampon (bounded buffer): bir mutex ve condition variable'larla halka tampon. İki condition variable (dolu değil, boş değil) mantığı okunur kılar. Öğeler değerle kopyalanır; bu yüzden aynı kuyruk int'leri, istekleri ve temaları taşıyabilir. while kullan: sahte uyanmalar (spurious wakeup) var ve başka bir thread öğeyi senden önce alabilir. queue_close'dan sonra pop kalan öğeleri döndürmeye devam eder, sonra başarısız olur; böylece worker'lar kuyruğu boşaltıp çıkar. push ise hemen başarısız olur; push'ta zaten bloke olmuş bir thread için de. close herkesi uyandırmalı: signal değil broadcast. Meşgul bekleme ve sleep yok. Projenin geri kalanı yokken kuyruğu tests/test_queue ile tek başına test edebilirsin.

GÖREV 2 · %25

Koreograf ve kaptan

Center

handle_plan_req, serve_request

- try_push; kuyruk doluysa: **BUSY 50**. Reader thread asla beklemez
- **Semafor**: en fazla --max-inflight planlayıcı çağrısı
- KV blokları Lab 6 havuzundan. Havuz **thread-safe değildir**: pool_mu ile sar
- Havuz boşsa: BUSY. Bu da bir geri basınçtır (backpressure)
- Her çıkış yolu blokları geri verir ve semaforu bırakır (post)

Kaptan

thinker_main

- Her THEME için: **PLAN_REQ** gönder, PLAN veya BUSY bekle
- BUSY: retry_ms + **rastgele** bir ek süre uyu. Sabit gecikme tüm leader'ları aynı anda tekrar denetir: livelock
- **validate_plan** çalıştır. Geçersizse: yeni seed, en fazla 6 deneme (--flaky 50 bunu test eder)
- Vazgeçsen bile her zaman **READY** gönder: topluluk yerinde bekler ve gösteri sürer

NOTLAR

Center'da bir reader thread PLAN_REQ alır ve asla beklememelidir: istek kuyruğuna try_push yapar, kuyruk doluysa hemen BUSY yanıtlar. Bir planlayıcı thread'i isteği çeker, bir semafor yuvası alır (aynı anda kaç planlayıcı çağrısı çalışacağını sınırlar) ve KV bloklarını Lab 6 havuzundan alır. Havuzun bilerek thread-safe değildir; bu yüzden her alloc ve free pool_mu altında yapılır. Havuz boşsa yanıt yine BUSY'dir: iki tür geri basınç (backpressure). Sonra plan_section çalışır (yavaş bir modeli taklit etmek için uyur), PLAN gönderilir; bloklar ve semafor her yolda serbest bırakılır. Leader'da thinker her tema için plan ister. BUSY'den sonra retry_ms artı rastgele bir ek süre bekler: tüm leader'lar aynı anda tekrar deneseydi tekrar tekrar çarpışırlardı; buna livelock denir. Gerçek bir model yanılabilir, bu yüzden kaptan bir plana asla güvenmez: doğrulayıcı hızı, çarpışmaları, bölgeyi ve yoğunluğu kontrol eder, geçersiz plan yeni bir seed ile yeniden istenir. flaky seçeneği planların yarısını bilerek yanlış yapar. Kaptan vazgeçse bile READY gönderilir; yoksa center o leader'ı sonsuza kadar bekler.

GÖREV 3 · %25

Birlikte adım atan dansçılar ve ızgara

Doluluk ızgarası

grid_claim, grid_release, grid_owner

- Hücre başına bir drone, her leader'ın bölgesinde
- İki mod: ızgara için **tek mutex** veya **satır başına bir mutex**
- Okuyucu da kilitlemeli. Neden?
- make bench ikisini 1, 2, 4, 8 thread ile ölçer. Tabloyu rapora koy ve **açıkla**

follower_main

sırayla her beat b için

```
BEAT b'yi bekle
hedef = plan veya bekle
release(eski hücre)
barrier
claim(hedef)
barrier
serial thread: POS satırları, ACK
```

Başarısız claim = güvenlik ihlali: say, logla, olduğun yerde kal.

NOTLAR

Doluluk ızgarası: tüm ızgara için tek mutex ile satır başına bir mutex. claim, release ve owner'ı iki mod için de yaz. Okuyucu da kilitlemeli: kilitsiz okuyucu yarım güncellenmiş bir durum görebilir; bu bir data race'tir. make bench, rastgele claim ve release işlemlerini her iki modda 1, 2, 4 ve 8 thread ile çalıştırır. Tabloyu raporuna koy ve kendi sayılarını açıkla: sayılar makinene ve çekirdeklerine bağlıdır (tek mutex üzerindeki çekişme, neyin ölçeklendiği ve darboğaz başka nerede olabileceği). Tempo 0'daki gösterinin süresini planlayıcının düşünme süresi belirler; bu yüzden orada ızgara darboğaz değildir. İkinci kısım: beat slaytındaki iki barrier ile follower_main. Geçerli bir planla claim asla başarısız olmaz; olursa bir güvenlik ihlalidir, o yüzden say ve logla.

GÖREV 4 VE 5 · HER BİRİ %10

Bulunacak bir deadlock, kaybedilecek bir kaptan

4 · Deadlock'u bul

```
./swapstress row 8 2
```

- grid_swap iki satırı kilitler. Satır modunda **takılır**
- Takılan thread'leri göster: gdb -p, thread apply all bt
- Hangi Coffman koşulunu bozduğunu söyle; kilit sıralamasıyla düzelt
- Çözümünün neden livelock'a yol açmadığını açıkla

5 · Kaptan düştü

```
./center --battery 1:20
```

- Poll ile değil, **pipe üzerindeki EOF** ile fark et. "captain 1 went dark" yazdır
- **waitpid**, pipe'ları kapat: zombie yok
- Saat döngüsünü (st_cv) uyandır ki beklemeyi bıraksın
- Yeniden başlat: ASSIGN, sonraki beat'ten itibaren THEME; topluluk evde başlar

NOTLAR

Görev 4: hazır verilen grid_swap iki hücreyi takas eder ve iki satırı da kilitler. İki thread aynı satır çiftini ters sırada takas ederse her biri bir kilidi tutar ve diğerini bekler: deadlock. swapstress bunu satır modunda saniyeler içinde yeniden üretir; global modda takılmaz. Takılan thread'leri gdb'de göster, bozduğun Coffman koşulunu söyle (döngüsel bekleme, kilit sıralamasıyla bozulur: her zaman önce küçük numaralı satırı kilitle) ve çözümünün neden livelock'a yol açmadığını açıkla. Görev 5: --battery 1:20 ile başlatılan leader, 20. beat başlarken çökmüş gibi çıkar. Center bunu pipe üzerindeki EOF ile fark etmeli, süreci waitpid ile toplamalı (zombie olmasın), pipe'ın iki ucunu da kapatmalı ve ondan ACK bekleyen saat döngüsünü uyandırmalı. Yeniden başlatma (varsayılan) ile bölge için yeni bir kaptan başlatılır: ASSIGN, sonra sonraki beat'ten itibaren THEME. Yeni topluluk evde başlar; bu yüzden hız denetimi bir sıçramayı affeder (skip_speed), ama çarpışmalar asla affedilmez. --no-restart ile bölge yalnızca gitmiş olarak işaretlenir. Raporda: dansçılar süreç olsaydı ne farklı olurdu? Bu Faz 3'ün konusu.

GÖREV 5, OLMASI GEREKEN GÖRÜNÜM

Kaptan 1 ölür, gösteri sürer



Beat 24: ikinci topluluk evine döndü (beyaz): yeni kaptan, yeni thread'ler.

Beat 25: yeni kaptanın planı başlar ve topluluk diğerleriyle dans eder.

Beat 55: resim yine oluşur; sıfır çarpışma ve sıfır hız ihlaliyle.

NOTLAR

Doğru bir Görev 5 böyle görünür (--battery 1:24 ile). Solda: beat 24'te, az önce ölen kaptana ait ikinci topluluk başlangıç sırasına geri döndü (beyaz noktalar): yeni bir kaptan başlatıldı ve follower'ları evde bekliyor. Ortada: bir beat sonra yeni kaptanın planı başlar ve topluluk yine diğerleriyle dans eder. Sağda: sonunda tüm resim tamamlanmıştır. Center her beat'i denetler: yeniden başlatmadan sonra eve sıçrama tam bir kez serbesttir, ama çarpışmalar asla serbest değildir.

BAŞLARKEN

Derle, test et ve çalışma sırası

```
cd labs/swarm-phase1-skeleton
make # derler, TODO'lar boş
make test # her kontrol için PASS/FAIL
make tsan # WARNING olmamalı
make bench # tek kilit ve satır kilitleri
./center --tempo 0 --ascii 8
```

Tam gösteri Görev 1–3'ün birlikte çalışmasını ister. O zamana kadar her parçayı tek başına test et: **test_queue**, **swapstress**, **gridbench**, veya protokol satırlarını elle yaz: **./leader --id 0**.

Önerilen sıra

1. queue.c
2. grid claim, release, owner
3. center: serve_request
4. leader: thinker
5. leader: followers
6. kaptanı yeniden başlatma
7. deadlock düzeltmesi

NOTLAR

Her şey iskelet repository'sinde; README.md yerleşimi açıklar. make, center, leader ve araçları derler; make test her kontrol için PASS veya FAIL yazdıran bir betik çalıştırır. Başlangıçta neredeyse her şey başarısız olur: beklenen durum. Gösteri ancak kuyruk, planlayıcı hattı ve follower'ların hepsi varsa çalışır; bu yüzden önerilen sırayla çalış ve parçaları ayrı ayrı test et: kuyruğun kendi birim testi var, ızgaranın swapstress ve gridbench araçları var ve bir leader'a protokol satırlarını elle yazabilirsin. Lab 6 kvpool.c ve arena.c dosyaların çalışıyorsa bunları src/ içine kopyala; Makefile hazır taklit yerine onları kullanır.

TESLİM VE NOTLANDIRMA

Ne teslim edilir, ne puan kırar

Teslim

- Repository'n (GitHub Classroom) ve Google Classroom'daki bağlantı
- **make test** ve **make tsan** geçer; Valgrind temiz
- tests/run_tests.sh içinde en az **iki kendi testin**
- **REPORT.md**: mimari taslağı, her mutex'in neyi koruduğu, kilit sıralaması, deadlock açıklaması, kaptan düşmesinin ele alınışı, bench tablosu, yapay zekâ yardımı

Sıfır tolerans

- Meşgul bekleme (busy-waiting)
- pthread_cond_wait etrafında **if**
- Korunmayan Lab 6 havuzu
- Kaptan ölünce kalan zombie süreçler
- Kapanışta takılma
- Eksik rapor

Bonus en fazla +%10: öncelikli kuyruk, ızgara için reader-writer lock, şarj istasyonu semaforlu batarya sayacı.

NOTLAR

Repository'ni GitHub Classroom ile teslim et ve bağlantıyı Google Classroom'a koy. make test ve make tsan geçmeli, Valgrind temiz olmalı. Test betiğine en az iki kendi testini ekle; örneğin daha fazla leader ya da iki kaptanı öldürmek. Rapor kısa ama zorunlu ve sadece anlatmamalı, açıklamalı. Yapay zekâ kullanımı serbesttir, ancak raporda belirtilmeli ve her satırı açıklayabilmelisin. Sıfır tolerans listesi, bir eşzamanlı program çalışıyor gibi görünse bile onu yanlış yapan şeylerdir. Bonus maddeleri föyün sonunda listelenmiştir.

TAKILDIĞINDA

Dört tuzak ve nasıl görülür

Kimsenin ulaşmadığı barrier

Barrier **tüm** follower'lar tarafından paylaşılır. İki bekleme arasında tek bir erken return veya continue, herkesi sonsuza kadar bekletir.

Hiç bırakılmayan kilit

Her unlock her yolda olmalı. Erken return'ler olağan sızıntıdır: kuyrukta ve serve_request'te.

Yanlış kilit sırası

Center'da: önce wlock, sonra st_mu. Aynı kural grid_swap'ı düzeltir: her zaman önce küçük numaralı satır.

Yarışı sleep ile gizlemek

Asla. Bunun yerine make tsan çalıştır ve yazdırdığı iki stack'i oku.

Takılma, --timeout saniye sonra **center: TIMEOUT** yazdırır. Şununla bağlan: `gdb -p PID`, sonra `thread apply all bt`.

BIL 322 · Swarm Faz 1 laboratuvarı

NOTLAR

Bu hafta ters gidecek dört şey. Bir: barrier bir leader'ın tüm follower'ları tarafından paylaşılır; biri iki bekleme arasında döngüden çıkarsa diğerleri sonsuza kadar bekler. İki: bir yolda bırakılmayan kilit, örneğin erken bir return; kodunu her return için oku. Üç: kilit sırası. Center'ın durum kilidi ile leader başına yazma kilidinin sabit bir sırası vardır ve center.c içinde yorumlanmıştır; bozmak deadlock yapabilir. Aynı fikir grid_swap deadlock'unu da düzeltir. Dört: sleep ile bir yarış durumunu gizlemek onu asla düzeltmez; çakışan iki erişimi yazdıran ThreadSanitizer'ı kullan. Gösteri takıldığında watchdog bir timeout yazdırır; sürece gdb ile bağlan ve tüm thread backtrace'lerini yazdır, kimin neyi beklediğini gör. Bu çıktı aynı zamanda Görev 4 açıklamasının kanıtıdır. Kuyrukla başla.